

# An optimized FPGA architecture for machine learning applications

Kareem Elsaid, M. Watheq El-Kharashi\*, Mona Safar

Computer and Systems Engineering Department, Ain Shams University, Cairo, Egypt

## ARTICLE INFO

### Keywords:

FPGA architectures  
Hardware accelerators  
IEEE-754 multipliers  
Machine learning applications  
Posit multipliers  
Verilog to Routing (VTR)

## ABSTRACT

FPGAs are currently the most suitable hardware accelerators to implement and accommodate the non-stop growth of machine learning applications. This paper presents an FPGA architecture with added posit multipliers that outweigh the current IEEE-754 multipliers in terms of delay and area. Since machine learning algorithms involve a lot of expensive mathematical operations, having such powerful multipliers in the proposed FPGA architecture will execute the needed operations with high efficiency, which will stand out for machine learning applications without compromising other FPGA applications. Experimental results using Verilog to Routing (VTR) on both machine learning and non-machine learning benchmarks have demonstrated that our proposed architecture consumes 10% less area than the Stratix IV FPGA. Furthermore, it consumes less power compared to both the Stratix IV and Stratix 10 FPGAs, with reductions of 22% and 10%, respectively. Additionally, our architecture outperforms the Stratix IV and Stratix 10 FPGAs in terms of critical path delay, with improvements of 25.5% and 43%, respectively.

## 1. Introduction

Nowadays, machine learning (ML) is utilized in almost all applications, where an outcome is precisely predicted without the need to be programmed to do the requirements using complex algorithms. These applications are becoming more and more complex, processing a huge amount of data [1]. Required data processing involves expensive mathematical operations and computation, which requires extensive parallelism and efficient hardware to implement with acceptable accuracy and speed.

Different high-performance hardware exists to accelerate ML algorithms, such as CPUs, GPUs, ASICs, and FPGAs [2]. ASICs provide the smallest area, while delivering the best speed and least power consumption, but they lack the reprogramming ability that exists in other hardware accelerators. FPGAs and CPUs/GPUs can be reconfigured based on intended applications, so they are more suitable for ML applications since ML algorithms are continuously evolving.

Commercially available GPUs, allow great parallelism and reconfigurable capabilities that suit ML applications. They lack speed compared to ASICs and consume more power. GPUs have limiting factors that impact negatively ML applications, such as the limited lifespan of the GPU, which spans around five years, and the fact that GPUs are not designed to withstand environmental factors, such as humidity and sun exposure. These limitations prevent GPUs from being used in automotive applications, for example. In contrast, FPGAs can withstand environmental factors and have longer lifespans.

FPGAs have better overall performance compared to GPUs [3]. Nurvitadhi et al. have observed better performance per Watt for FPGAs over GPUs, when implementing binarized or recurrent neural networks with and without batching of input samples [2]. Asano et al. also observed better performance for FPGA over GPU, when implementing image processing problems using the k-means clustering algorithm, owing to the massive parallelism provided by the FPGA [4].

The recently mentioned advantages of FPGAs over other hardware accelerators make them attractive and widely used in different fields, including aerospace, military applications, automotive applications, communications, network security, and ML applications [5–7]. FPGA vendors have made major enhancements to their shipped FPGA devices over the years. FPGAs are no longer a sea of programmable soft logic, such as Logic Elements (LEs) and Configurable Logic Blocks (CLBs). Hard logic like integrated memory and Digital Signal Processing units (DSPs) are included in current FPGA architectures [8].

The latest FPGA devices, such as Agilex from Intel, include an ARM process subsystem [9]. All of these new subsystems and hard blocks are there to accommodate the huge designs targeted on the FPGA fabric. Currently, around 63% of the projects target FPGA devices with one or more embedded processors to load their designs [10]. Furthermore, ML algorithms involve a lot of matrix multiplication and addition, which will be efficiently handled in hard DSP blocks added to the FPGA fabric instead of using the FPGA soft logic, which will be much slower and consume a larger area. For instance, the softmax layer, which is

\* Corresponding author.

E-mail addresses: [G18092909@eng.asu.edu.eg](mailto:G18092909@eng.asu.edu.eg) (K. Elsaid), [watheq.elkharashi@eng.asu.edu.eg](mailto:watheq.elkharashi@eng.asu.edu.eg) (M.W. El-Kharashi), [mona.safar@eng.asu.edu.eg](mailto:mona.safar@eng.asu.edu.eg) (M. Safar).

one of the deep neural network layers, involves expensive arithmetic operations [11].

In this paper, we present a novel FPGA architecture with hard-positt multipliers optimized for ML applications. Unlike traditional approaches that add hard blocks to handle ML-related tasks, we focused on optimizing the soft logic and DSP blocks, which are commonly used in almost all FPGA-targeted applications, to maintain flexibility. Our FPGA architecture is the first to incorporate posit hardware units, which leads to higher operational frequencies, lower power consumption, and reduced area.

CAD tools, such as Verilog to Routing (VTR) and OpenFPGA are utilized to simulate and investigate different FPGA architectures [12–14]. VTR is specifically chosen in this work because of its credibility and the availability of a wide range of benchmarks for assessing architectures in various domains [12,14]. VTR is an open-source project designed for the exploration of FPGA architecture concepts and the assessment of their performance against standard benchmark designs. In our work, we employed VTR to assess the proposed architecture in comparison to commercially available FPGA architectures provided by VTR. VTR allows users to input their circuits in Verilog HDL and define the FPGA architecture using XML. The VTR workflow comprises three main stages: elaboration, placement and routing, and timing analysis. VTR also offers a variety of benchmarks and flagship FPGA architectures to support research and the modeling of commercial FPGAs.

The following are the primary contributions of this work:

- Incorporating posit multipliers in the DSP block, yielding a higher frequency of operation, less power consumed, and less area
- Improving the FPGA logic cluster for intensive arithmetic operations while maintaining flexibility for other applications
- Evaluating the proposed architecture with Koios benchmarks [15] and other VTR8 benchmarks [12] against Stratix IV and Stratix 10 FPGAs for ML and non-ML applications

The following is a breakdown of the paper's content. Related work is covered in Section 2. The proposed FPGA architecture is discussed in Section 3. The experimental findings are presented in Section 4. Section 5 summarizes the findings as well as future work.

## 2. Related work

Within this section, we will conduct a review of previous research efforts pertaining to FPGA architecture enhancements for ML applications. Furthermore, we will explore recent developments in posit multipliers.

### 2.1. FPGA architecture enhancements for ML

Numerous designs have been suggested and put into practice to enhance the performance of AI/ML algorithms, with a primary emphasis on augmenting computational resources, as well as improving memory bandwidth and efficiency. Advancements to the FPGA to enhance its performance for ML applications can be done through direct changes in the FPGA architecture itself or indirect overlay acceleration elements that are used in conjunction with the FPGA itself.

Arora et al. have proposed an FPGA architecture that has hard blocks of matrix multipliers [16]. These matrix multipliers were added to the FPGA architecture in conjunction with other hard blocks in their architecture, such as RAM blocks, CPUs, and DSP blocks. They have tested the architecture using VTR flow against Stratix 10 FPGAs and found that their architecture has area and delay savings.

Achronix has produced Speedster7t FPGAs based on TSMC 7 nm technology node targeting ML applications [17]. The core of the FPGA is comprised of embedded memories, 6-input LUTs, and ML processors (MLP). MLPs consist of register files, RAM blocks, and a fracturable adder and accumulator.

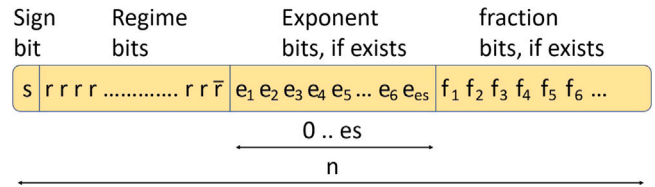


Fig. 1. Posit format.

Although both of the discussed architectures make ML applications faster and provide area savings, this compromises the flexibility of the FPGA in non-ML applications due to adding dedicated resources to the FPGA for machine-learning-related applications only.

Xilinx has Versal products that include a dedicated artificial intelligence (AI) engine array on the FPGA fabric as the programmable logic [18,19]. AI engines are classified into two types: AI Engine and AI Engine ML, both of which provide significant performance enhancements over previous FPGAs in areas, such as, ML applications and signal processing.

Intel has used an embedded multi-die interconnect bridge to provide a flexible way to have a multi-chip integration of FPGAs and ASICs in a single package [20]. The embedded multi-die interconnect bridge is used in their latest FPGAs and work was done to target ML applications by connecting TensorTile ASICs to Stratix10 FPGAs, enhancing the performance of the FPGA-ASIC package in matrix and vector operations [21].

### 2.2. Posit multipliers

The IEEE-754 floating-point number system faces several issues, including reserved bit patterns for NaNs, violations of the associative property in some cases [22], overflow and underflow in unary operations, such as the squaring of a floating number [23], leading to inefficient area and power usage in floating-point units [24]. In response, an alternative what called “posit” has been proposed by Gustafson, which overcomes these limitations and offers a larger dynamic range and improved accuracy [23]. However, hardware support for posits may consume more area and power due to their configurability, but there are ways to mitigate these issues, which will be discussed later in this paper. For more details about the posit format, shown in Fig. 1, we refer the reader to [23].

Chaurasiya et al. implemented a parameterized posit arithmetic unit generator and compared it to floating-point units. They discovered that it had similar propagation delays and areas but was much more accurate with posit hardware units [24]. Shekhawat et al. have proposed a posit multiplier that was tested using Xilinx Virtex-7, where an approximately 10% reduction in propagation delay was observed compared to floating-point units [25].

Vinotheni et al. have designed a posit multiplier and did a comparative analysis of area, power, and delay against floating-point multipliers and found that the posit multipliers were outperforming in the three domains, especially in the power domain where 26.13% less dynamic power was consumed by the posit multiplier [26]. This experiment was done on Virtex-7 FPGA. Norris et al. have proposed an approximate and iterative posit multiplier that provides a higher dynamic range but with more hardware resources used compared to the iterative floating-point hardware units after testing the multiplier using Xilinx UltraScale+ FPGA [27].

The number of regime and exponent bits in posit hardware significantly affect its performance and resource usage. Gohil et al. addressed this issue by designing a fixed-posit multiplier unit with specific regime and exponent bit values, which was synthesized for 28 nm technology using Synopsys Design Compiler (DC) [28]. When compared to the IEEE-754 floating-point multiplier, their solution exhibited impressive results, including up to 66% reduction in area, a 26% decrease in delay, and a 70% reduction in power consumption.

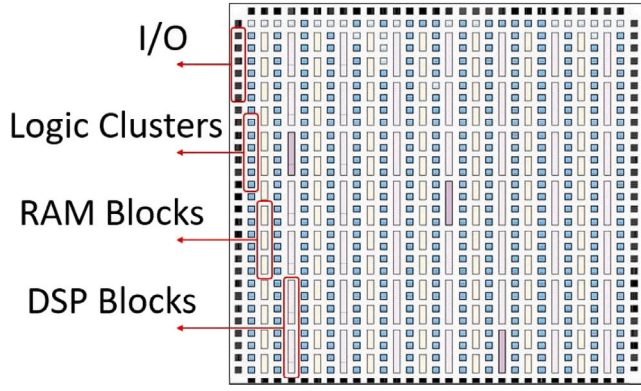


Fig. 2. Overview of the proposed FPGA architecture.

### 3. Proposed FPGA architecture

The proposed architecture is a column-based architecture, where the Input/Output pads are at the edges. RAMs and DSPs using posit multipliers are the FPGA hard blocks. Fig. 2 shows the proposed architecture, where columns of logic clusters and RAM blocks are followed by columns of logic clusters and DSP blocks, forming a square-like FPGA.

#### 3.1. Logic cluster architecture

Each logic cluster has 10 Adaptive Logic Modules (ALMs) with a total of 80 inputs and 40 outputs in which various arithmetic, logic, and memory functions can be implemented. Ahmed and Rose conducted extensive testing to determine the number of ALMs that will produce the optimum FPGA performance. They concluded that a logic cluster with 3 to 10 ALMs is the optimal number [29]. Nonetheless, different logic clusters and LUT sizes were tested and a cluster size of 10 and a fracturable LUT of size 6 were found to produce the best delay-area product, confirming the results previously found in FPGA research [29]. Each ALM has eight inputs and four outputs, with three modes of operation available: normal, extended LUT, and arithmetic modes.

Figs. 3 and 4 show the LUT configurations in the normal and extended modes of operation. Depending on the FPGA configurations and intended functionality, the normal mode of operation supports 6-LUT, 5-LUT, 4-LUT, or 3-LUT configurations. When using the typical 6-LUT configuration, there are two logically equivalent outputs connected to the LUT, either a combinational output or a registered output, depending on the implemented circuit type. The remaining 2 outputs of the ALM are connected directly to the remaining 2 inputs through flip-flops that provide registered inputs. When slicing the 6-LUT down into several smaller LUTs, input sharing is handled accordingly, utilizing all the available inputs. Extended LUT mode utilizes all the available inputs into one big 8-LUT.

Fig. 5 shows the arithmetic mode, which uses four 4-LUTs and four adders with two levels of carry chains used with arithmetic operations. The LUTs perform pre-addition operations, and the adders do the additions. Dafrawy et al. tested this architecture for ML applications and found that adding these adders optimized the netlist compared to commercial ALM architectures without sacrificing the ALM's flexibility [30].

#### 3.2. Local routing architecture

The local interconnect inside the logic cluster is a series of multiplexers responsible for the local routing of inputs to the LUTs, as shown in Fig. 6. The global routing tracks, direct links from horizontal

neighboring clusters, and ALM outputs with sparse (50%) population levels are the three channels through which local interconnects receive inputs.

Direct links are horizontal direct connections from the output of the ALM to the neighboring block implemented on a higher metal layer without feeding the global tracks. The use of direct links facilitates fast propagation across neighboring blocks. Each logic cluster can drive half of its outputs on a higher metal layer to the adjacent cluster and receive half the outputs of the adjacent cluster in its local interconnect. Carry chains have dedicated carry chain routing wires that connect the ALMs of the same logic cluster to build multi-bit adders. Wider adders are built efficiently with dedicated vertical carry chains routing wires that connect adjacent logic clusters.

#### 3.3. FPGA hard blocks

FPGA hard blocks can speed up arithmetic operations but affect the flexibility of the FPGA. Arora et al. presented dedicated matrix multipliers as hard blocks on the FPGA for ML applications, which will speed up the targeted applications but limit the FPGA's performance in other applications [16]. Our architecture adds RAM and DSP blocks as flexible units for different applications. DSP blocks have added posit multipliers to improve ML applications' performance over IEEE-754 multipliers.

The architecture has 20 KB memory blocks. RAMs can be set up as simple dual-port, as well as true dual-port RAMs. Both simple and true dual-port modes can be configured into  $1024 \times 20$  and  $2048 \times 10$  modes. The simple dual-port mode has an added  $512 \times 40$  mode.

The DSP block can operate in fixed-point or floating-point modes, supporting single-precision and half-precision addition, multiplication, and accumulation operations. Also, the DSP can be fractured to accommodate low-precision fixed-point arithmetic, for int4 and int9 multiply and MAC operations. This fragmentation is done while maintaining the existing DSP block routing interface, which is essential for supporting ML workloads. Previously, we mentioned that posit hardware may have a bigger area waste than floating-point hardware. This can be mitigated by fixing the allocated bits dedicated to each field in the posit number. This was done by Gohil et al. where different fixed values for posit multipliers were tested, and in terms of delay, size, and power usage, posit multipliers beat IEEE-754 floating-point multipliers in various ML applications [28], so this multiplier is a perfect fit for our FPGA architecture and is used in the DSP block. This multiplier was synthesized and evaluated at 28 nm using DC. Therefore, we have proposed our architecture to run on a 28 nm technology node, where scaling equations are used to map different delay and area values to the 28 nm technology node [31].

#### 3.4. Global routing architecture

The proposed architecture has an island-style architecture, where global interconnects meet and connect through switch blocks, as shown in Fig. 7. The routing density is uniform across the architecture, with a routing channel width of  $W = 300$ . There are two different wiring segment lengths,  $L = 4$  and  $L = 16$ , with 260 wires in length 4 and 40 in length 16. The routing channel width is big enough to handle large circuits [14,32].

L16 wires are used mainly for quick routing across different FPGA blocks and implemented on high metal layers where they are not directly connected to block pins; instead, they are connected to switch blocks every four clusters due to the expensive via connections done with block pins and L4 wires, which are implemented at a lower metal layer.

Multiplexers serve as connection blocks between the routing tracks and the logic clusters with 50% populated sparse crossbar. The connection block flexibility for inputs  $F_{c_{in}} = 0.15$ , and the connection block

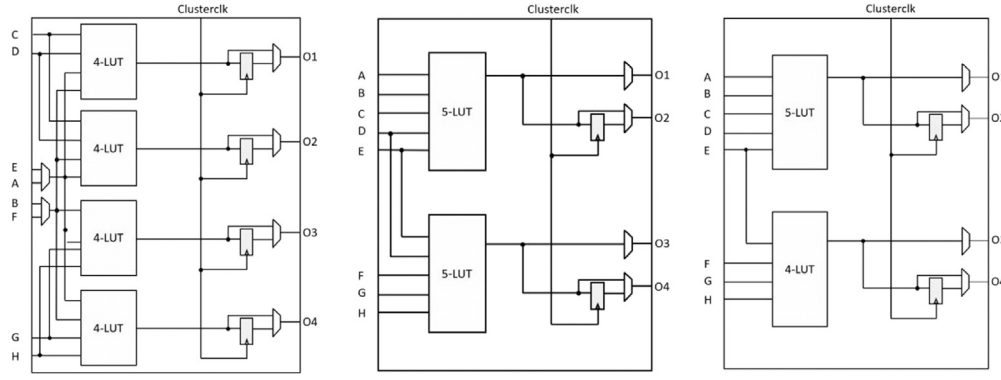


Fig. 3. 4-LUT, 5-LUT and 5-4-LUT configurations.

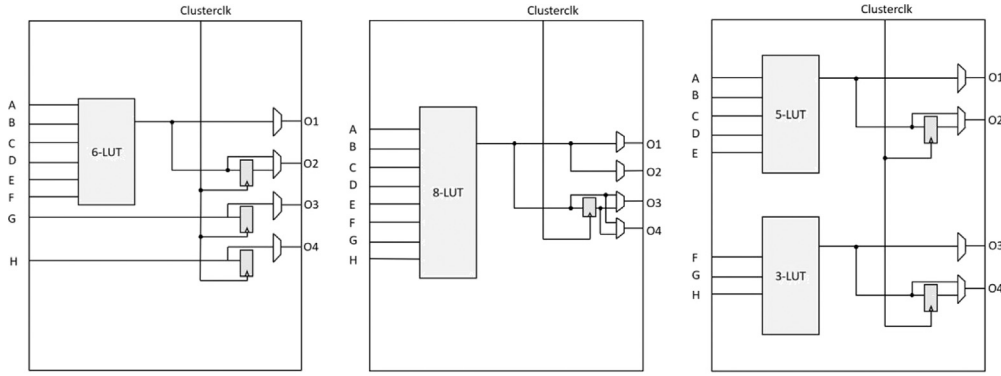


Fig. 4. 6-LUT, 8-LUT, 4-LUT and 5-3-LUT configurations.

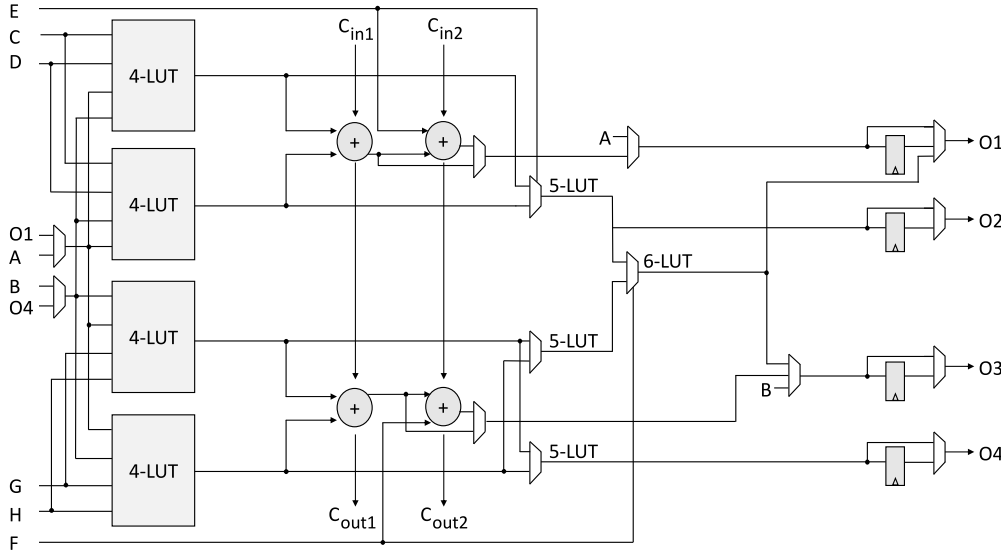


Fig. 5. ALM in arithmetic mode.

flexibility for outputs  $F_{c_{out}} = 0.1$ . These connection block flexibility values are as per the latest guidelines for FPGA routing parameters [8].

Logic cluster pins are located on three sides of the cluster, as shown in Fig. 7. Different switch blocks were tested, and the Wilton switch block pattern was adopted, giving the optimum area-delay product as expected from previous research on switch block flexibility,  $F_s = 3$ , allowing the signal to turn right, left, or straight [33,34]. There was no benefit to increasing the switch block flexibility by more than 3, as per the research in [8,35].

#### 4. Experimental results

VTR ships FPGA architectures that closely model commercially available FPGAs. We will use the shipped Stratix IV and Stratix 10 FPGA architectures to compare the performance of the proposed architecture to them. VTR has a set of benchmarks that are shipped to be used according to the FPGA application needed. Koios benchmarks will be used since they are targeting ML applications research [15]. Seven Koios benchmarks will be used to test the proposed architectures, which are the following:



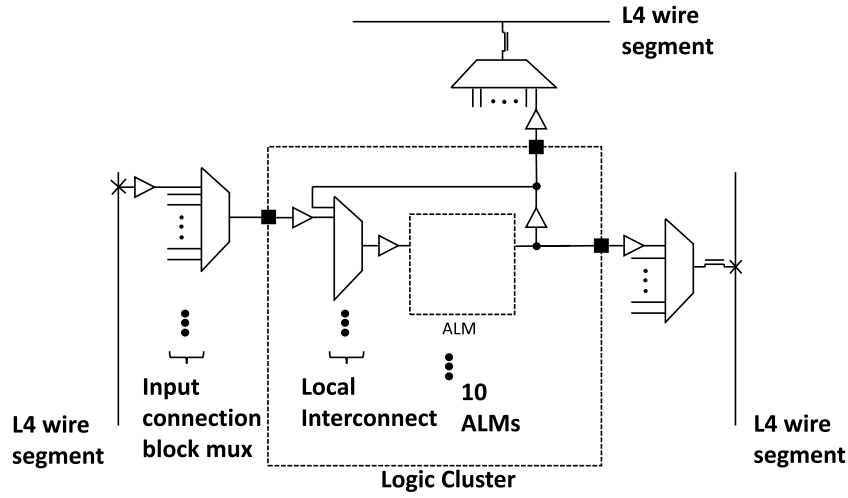


Fig. 6. Logic cluster local interconnect.

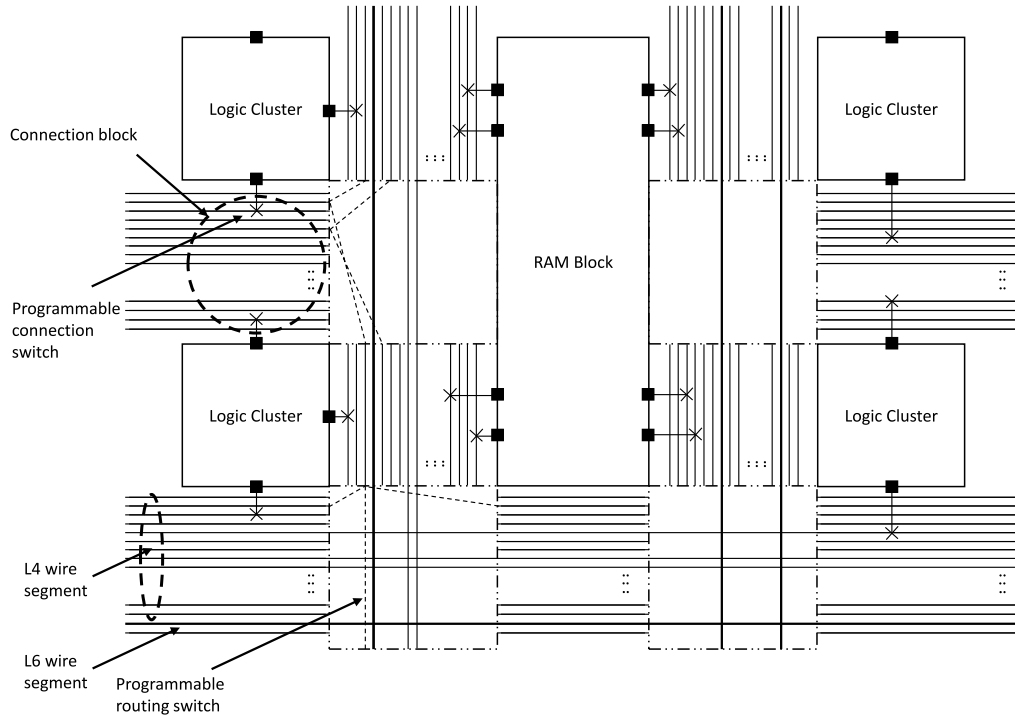


Fig. 7. Detailed routing architecture of the proposed architecture.

- clstm\_like (CLSTM-like accelerator) based on [36]
- tpu\_like (Google-TPU-v1-like accelerator) based on [37]
- softmax based on [38]
- conv\_layer (GEMM-based convolution)
- eltwise\_layer (Matrix elementwise add/sub/mult)
- bnn (4-layer binary neural network) based on [39]
- dla\_like (Intel-DLA-like accelerator) based on [40]

Non-ML benchmarks are also shipped by VTR for FPGA research in different domains. Five of these benchmarks will be used to test the proposed architecture to evaluate its flexibility and performance against non-ML applications, which are the following:

- mcml based on [41] (Domain: Medical Physics)
- LU32PEEng (Domain: Math)
- bgm based on [42] (Domain: Finance)
- arm\_core (Domain: Soft processor)

- blob\_merge (Domain: Finance)

The benchmarks utilized in this work were selected to test the proposed architecture through its limits; for example, the most DSP-intensive Koios benchmarks are clstm\_like and tpu\_like [15]. These benchmarks will test the handling of the arithmetic operations as well as how fast to output the results of these operations. Also, the two largest ML benchmarks in the Koios family are clstm\_like, and dla\_like, while mcml and LU32PEEng are the largest non-ML benchmarks shipped. These will test the overall limitations of the architecture in terms of area and frequency.

VTR calculates the area for each block in terms of MWTAs, but a recent study found that it does not provide an accurate estimate of the active area being utilized when compared to the actual layout of the same FPGA blocks, which is why we are comparing the number of blocks utilized [43].

**Table 1**  
Delay, power, and area comparison between ML benchmarks.

| Benchmark     | FPGA Architecture     | Parameters         |                         |                   |           |
|---------------|-----------------------|--------------------|-------------------------|-------------------|-----------|
|               |                       | Critical path (ns) | Power (mW) <sup>a</sup> | Block utilization |           |
|               |                       |                    |                         | Logic cluster     | DSP block |
| softmax       | Proposed Architecture | 8.96               | 267                     | 1005              | 800       |
|               | Stratix IV            | 9.81               | 371                     | 1166              | 8         |
|               | Stratix 10            | 9.61               | 334                     | 1009              | 8         |
| conv_layer    | Proposed Architecture | 6.1                | N/A                     | 1183              | 84        |
|               | Stratix IV            | 7.55               | N/A                     | 1365              | 84        |
|               | Stratix 10            | 9.54               | N/A                     | 1182              | 84        |
| eltwise_layer | Proposed Architecture | 6.8                | N/A                     | 709               | 6         |
|               | Stratix IV            | 5.16               | N/A                     | 902               | 6         |
|               | Stratix 10            | 7.85               | N/A                     | 717               | 12        |
| tpu_like      | Proposed Architecture | 19.95              | N/A                     | 4254              | 1064      |
|               | Stratix IV            | 42.42              | N/A                     | 5215              | 1064      |
|               | Stratix 10            | 55.85              | N/A                     | 4258              | 1072      |
| clstm_like    | Proposed Architecture | 19.41              | N/A                     | 9309              | 362       |
|               | Stratix IV            | 26.91              | N/A                     | 10311             | 361       |
|               | Stratix 10            | 36.81              | N/A                     | 9325              | 362       |
| dla_like      | Proposed Architecture | 6.88               | N/A                     | 5842              | 128       |
|               | Stratix IV            | 9.17               | N/A                     | 6282              | 128       |
|               | Stratix 10            | 11.18              | N/A                     | 5851              | 128       |
| bnn           | Proposed Architecture | 7.36               | N/A                     | 5688              | 63        |
|               | Stratix IV            | 8.56               | N/A                     | 5997              | 63        |
|               | Stratix 10            | 9.16               | N/A                     | 5691              | 63        |

<sup>a</sup> Power measurements are not available for multi-clock domain designs due to a VTR limitation.

**Table 2**  
Delay, power, and area comparison between non-ML benchmarks.

| Benchmark (Domain)        | FPGA Architecture     | Parameters         |            |                   |           |
|---------------------------|-----------------------|--------------------|------------|-------------------|-----------|
|                           |                       | Critical path (ns) | Power (mW) | Block utilization |           |
|                           |                       |                    |            | Logic cluster     | DSP block |
| mcml (Medical Physics)    | Proposed Architecture | 114.08             | 1599       | 6797              | 106       |
|                           | Stratix IV            | 151.91             | 2132       | 7078              | 27        |
|                           | Stratix 10            | 230.23             | 1919       | 6795              | 106       |
| arm_core (Soft Processor) | Proposed Architecture | 29.03              | 146        | 836               | 0         |
|                           | Stratix IV            | 34.44              | 165        | 1008              | 0         |
|                           | Stratix 10            | 37.79              | 143        | 839               | 0         |
| bgm (Finance)             | Proposed Architecture | 45.24              | 330        | 2123              | 11        |
|                           | Stratix IV            | 58.36              | 453        | 2712              | 11        |
|                           | Stratix 10            | 75.67              | 398        | 2121              | 11        |
| LU32PEEng (Maths)         | Proposed Architecture | 62.47              | 1680       | 5879              | 32        |
|                           | Stratix IV            | 68.73              | 2400       | 7495              | 32        |
|                           | Stratix 10            | 79.99              | 2208       | 5882              | 32        |
| blob_merge (Finance)      | Proposed Architecture | 9.14               | 80         | 533               | 0         |
|                           | Stratix IV            | 13.31              | 91         | 619               | 0         |
|                           | Stratix 10            | 14.25              | 84         | 538               | 0         |

Tables 1 and 2 show the block utilization, power consumption, and critical path delay comparisons. We can observe that the proposed architecture has used fewer logic clusters against all the benchmarks tested when compared to Stratix IV. Also, there are no significant differences between the number of DSP blocks used in the 3 architectures. Since the DSP block size is a multiple of four of the logic cluster size in all architectures, we can calculate the total number of blocks used to implement each benchmark. The proposed architecture offers, on average, a 10% area saving over Stratix IV and a non-significant area saving over Stratix 10. The proposed architecture offers area savings for the different modes supported by the DSP block and more efficient LUT configurations in the ALM.

It was observed that during the implementation of arm core and blob merge benchmarks on all architectures, only logic clusters were utilized. The proposed architecture uses fewer logic clusters due to its more efficient arithmetic mode LUT configuration, offering 17.06% and 13.90% area savings over Stratix IV against arm core and blob merge, respectively.

Multi-clock domain designs lack power data in Table 1 as power analysis in VTR is limited to single-clock domain designs. The reported power values include both static and dynamic power. The proposed architecture, on average, consumes 22% less power than the Stratix IV and achieves 14% power savings compared to the Stratix 10. For the designs that do use DSP blocks, significant power savings are observed. Averaging the power savings for these designs shows improvements of 28% over the Stratix IV and 20% over the Stratix 10. Notably, the power savings are attributed to the posit multiplier used in the DSP. Gohil et al. emphasized the significant power-saving advantage in their research on posit multipliers [28]. This highlights the power-saving capabilities of the DSP block design in our architecture that incorporates posit multipliers, particularly when used in arithmetic-heavy designs.

Table 3 shows the average critical path delay improvement of the proposed architecture against ML and non-ML benchmarks. The proposed architecture outperforms Stratix IV and Stratix 10 in terms of having less critical path delays in almost all benchmarks, with more delay savings in ML benchmarks. Tables 1 and 2 show that the

**Table 3**  
Average critical path delay comparison.

|                        | Average improvement (%) |                   |
|------------------------|-------------------------|-------------------|
|                        | ML benchmarks           | Non-ML benchmarks |
| Proposed vs Stratix 10 | 46                      | 40                |
| Proposed vs Stratix IV | 31                      | 20                |

proposed architecture has an average critical path delay improvement of 25.5% against Stratix IV and 43% against Stratix 10 across all domains, proving its flexibility. This delay reduction is due to faster posit multipliers in DSP blocks and routing architecture parameters, per the latest FPGA research [8,28].

Finally, all architectures implemented `arm_core` and `blob_merge` using logic clusters only. These benchmarks can be used to judge whether the arithmetic mode in the logic clusters implemented is effective at handling expensive arithmetic operations or not. We can see that the proposed architecture saves 15.71% in delay over Stratix IV and 23.18% over Stratix 10 for `arm_core` benchmark. In addition, the proposed architecture saves 31.30% in delay over Stratix IV and 35.9% over Stratix 10 for the `blob_merge` benchmark. This is due to added adders inside logic clusters in arithmetic mode and carry chains connecting logic clusters for fast carry propagation.

## 5. Conclusion and future work

This paper proposes an FPGA architecture optimized for ML applications while keeping the FPGA's flexibility. This was achieved by enhancing the DSP block with posit multipliers, which can handle expensive operations better than IEEE-754 floating-point multipliers, and by using the latest research findings in routing architecture and logic cluster, such as the more efficient ALM architecture in arithmetic mode tested with ML applications. VTR compared the proposed FPGA architecture to Stratix IV and 10 FPGAs. The architectures were compared using seven ML and five non-ML benchmarks. Power, area, and delay were measured against these benchmarks. The proposed architecture outperforms both the Stratix IV and Stratix 10 in terms of critical path delay, with improvements of 25.5% and 43%, respectively. Additionally, it consumes 10% less area than the Stratix IV. Moreover, the proposed architecture achieves power savings of 22% and 14%, when compared to Stratix IV and Stratix 10, respectively. In this work, the latest research findings were used to propose a flexible FPGA architecture for ML applications.

Finally, routing architecture enhancements should be the next focus of exploration since routing architecture and routing channels are the main contributors to the FPGA area. The proposed architecture can be tested against different tools, such as OpenFPGA instead of using VTR only to get more reliable data.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

No data was used for the research described in the article.

## Acknowledgments

We would like to acknowledge that a preliminary version of this work has been published in the Proceedings of the International Conference on Microelectronics (ICM) in 2022 [44]. We thank the conference organizers for providing valuable feedback on our initial findings, which helped refine our research and improve the final version of this work.

## References

- [1] Sarker IH. Machine learning: Algorithms, real-world applications and research directions. *SN Comput Sci* 2021;2(3).
- [2] Nurvitadhi E, Sim J, Sheffield D, Mishra A, Krishnan S, Marr D. Accelerating recurrent neural networks in analytics servers: Comparison of FPGA, CPU, GPU, and ASIC. In: 2016 26th International conference on field programmable logic and applications. 2016.
- [3] Hu Y, Liu Y, Liu Z. A survey on convolutional neural network accelerators: GPU, FPGA and ASIC. In: 2022 14th International conference on computer research and development. 2022, p. 100–7.
- [4] Asano S, Maruyama T, Yamaguchi Y. Performance comparison of FPGA, GPU and CPU in image processing. In: 2009 International conference on field programmable logic and applications. 2009, p. 126–31.
- [5] Hoozemans J, Peltenburg J, Nonnemacher F, Hadnagy A, Al-Ars Z, Hofstee HP. FPGA acceleration for big data analytics: Challenges and opportunities. *IEEE Circuits Syst Mag* 2021;21(2):30–47.
- [6] Sobhanmanesh F, Nooshabadi S, Aboutanios E. An efficient VLSI architecture for  $4 \times 4$  16-QAM sorted QR-factorisation based V-BLAST decoder. *AEU Int J Electron Commun* 2011;65(5):443–52.
- [7] Farghaly SH, Ismail SM. Floating-point discrete wavelet transform-based image compression on FPGA. *AEU Int J Electron Commun* 2020;124:153363.
- [8] Boutros A, Betz V. FPGA architecture: Principles and progression. *IEEE Circuits Syst Mag* 2022;21:4–29.
- [9] Chromczak J, Wheeler M, Chiasson C, et al. Architectural enhancements in Intel® Agilex™ FPGAs. In: 2020 ACM/SIGDA international symposium on field-programmable gate arrays. 2020.
- [10] Foster HD. 2018 FPGA functional verification trends. In: 2018 19th International workshop on microprocessor and SOC test and verification. IEEE; 2018, p. 40–5.
- [11] Alabassy B, Safar M, El-Kharashi MW. A high-accuracy implementation for softmax layer in deep neural networks. In: 2020 15th Design & technology of integrated systems in nanoscale era. 2020.
- [12] Murray KE, Petelin O, Zhong S, Wang JM, Eldafrawy M, et al. VTR 8: High-performance CAD and customizable FPGA architecture modelling. *ACM Trans Reconfigurable Technol Syst* 2020;13(2).
- [13] Tang X, Giacomini E, Chauviere B, Alacchi A, Gaillardon P-E. OpenFPGA: An open-source framework for agile prototyping customizable FPGAs. *IEEE Micro* 2020;40(4):41–8.
- [14] Luu J, Goeders J, Wainberg M, Somerville A, Yu T, Nasartschuk K, et al. VTR 7.0: Next generation architecture and CAD system for FPGAs. *ACM Trans Reconfigurable Technol Syst* 2014;7(2).
- [15] Arora A, Boutros A, Rauch D, Rajen A, Borda A, et al. Koios: A Deep Learning Benchmark Suite for FPGA Architecture and CAD Research. In: 2021 31st International conference on field-programmable logic and applications. IEEE; 2021, p. 355–62.
- [16] Arora A, Wei Z, John LK, Hamamu: Specializing FPGAs for ML applications by adding hard matrix multiplier blocks. In: 2020 IEEE 31st international conference on application-specific systems, architectures and processors. IEEE; 2020, p. 53–60.
- [17] Cairncross A, Henry B, Chalmers C, Reid D, Shipton J, Fowler J, et al. AI benchmarking on achronix speedster7t FPGAs. 2023.
- [18] Ahmad S, Subramanian S, Boppana V, Lakka S, Ho F-H, Knopp T, et al. Xilinx first 7nm device: Versal AI core (VC1902). In: Hot chips symposium. 2019.
- [19] Vissers K. Versal: The Xilinx adaptive compute acceleration platform (acap). In: Proceedings of the 2019 ACM/SIGDA international symposium on field-programmable gate arrays. 2019, p. 83–83.
- [20] Mahajan R, Sankman R, Patel N, et al. Embedded Multi-die Interconnect Bridge (EMIB) – a high density, high bandwidth packaging interconnect. In: 2016 IEEE 66th electronic components and technology conference. 2016, p. 557–65.
- [21] Nurvitadhi E, Cook J, Mishra A, et al. In-Package Domain-Specific ASICs for Intel® Stratix® 10 FPGAs: A Case Study of Accelerating Deep Learning Using TensorTile ASIC. In: 2018 28th International conference on field programmable logic and applications. 2018, p. 106–1064.
- [22] Lafage V. Revisiting what every computer scientist should know about floating-point arithmetic. 2020, arXiv preprint arXiv:2012.02492.
- [23] Gustafson JL, Yonemoto IT. Beating floating point at its own game: Posit arithmetic. *Supercomput Front Innov* 2017;4(2):71–86.
- [24] Chaurasiya R, Gustafson, et al. Parameterized posit arithmetic hardware generator. In: 2018 IEEE 36th international conference on computer design. IEEE; 2018, p. 334–41.
- [25] Shekhawat D, Jangir A, Pandey JG. A hardware generator for posit arithmetic and its FPGA prototyping. In: 2021 25th International symposium on VLSI design and test. IEEE; 2021.
- [26] Vinotheni M, Karthika K. Implementation of high performance Posit-Multiplier. *Int J Eng Technol Manag Sci* 2023.
- [27] Norris CJ, Kim S. An approximate and iterative posit multiplier architecture for FPGAs. In: 2021 IEEE international symposium on circuits and systems. IEEE; 2021.
- [28] Gohil V, Walia S, Mekie J, Awasthi M. Fixed-posit: A floating-point representation for error-resilient applications. *IEEE Trans Circuits Syst II* 2021;68(10):3341–5.

- [29] Ahmed E, Rose J. The effect of LUT and cluster size on deep-submicron FPGA performance and density. *IEEE Trans Very Large Scale Integr (VLSI) Syst* 2004;12(3):288–98.
- [30] Eldafrawy M, Boutros A, Yazdanshenas S, Betz V. FPGA logic block architectures for efficient deep learning inference. *ACM Trans Reconfigurable Technol Syst (TRET)* 2020;13(3).
- [31] Stillmaker A, Baas B. Scaling equations for the accurate prediction of CMOS device performance from 180nm to 7nm. *Integration* 2017;58:74–81.
- [32] Petelin O, Betz V. The speed of diversity: Exploring complex FPGA routing topologies for the global metal layer. In: 2016 26th International conference on field programmable logic and applications. 2016.
- [33] Tang X, Giacomini E, Alacchi A, Gaillardon P-E. A study on switch block patterns for tileable FPGA routing architectures. In: 2019 International conference on field-programmable technology. IEEE; 2019, p. 247–50.
- [34] Mukherjee S, Roy S. Testing the effect of different switch box architectures on detailed routing in FPGA. In: International conference on communication, circuits, and systems, no. 5. 2013.
- [35] Schmit H, Chandra V. FPGA switch block layout and evaluation. In: ACM/SIGDA tenth international symposium on field-programmable gate arrays. 2002, p. 11–8.
- [36] Wang S, Li Z, Ding C, Yuan B, Qiu Q, Wang Y, et al. C-LSTM: Enabling efficient LSTM using structured compression techniques on FPGAs. In: 2018 ACM/SIGDA international symposium on field-programmable gate arrays. 2018, p. 11–20.
- [37] Jouppi NP, Young C, et al. In-datacenter performance analysis of a tensor processing unit. In: Proceedings of the 44th annual international symposium on computer architecture. 2017.
- [38] Wei Z, Arora A, Patel P, John L. Design space exploration for softmax implementations. In: 2020 IEEE 31st international conference on application-specific systems, architectures and processors. IEEE; 2020, p. 45–52.
- [39] Guglielmo GD, Duarte JM, Harris PC, Hoang DA, et al. Compressing deep neural networks on FPGAs to binary and ternary precision with HLS4ML. *Mach Learn: Sci Technol* 2020;2.
- [40] Boutros A, Yazdanshenas S, Betz V. You cannot improve what you do not measure. *ACM Trans Reconfigurable Technol Syst (TRET)* 2018;11.
- [41] Luu J, Redmond K, Lo W, Chow P, Lilge L, Rose J. FPGA-based Monte Carlo computation of light absorption for photodynamic cancer therapy. In: 2009 17th IEEE symposium on field programmable custom computing machines. IEEE; 2009, p. 157–64.
- [42] Zhang G, Leong PHW, et al. Reconfigurable acceleration for Monte Carlo based financial simulation. In: 2005 IEEE international conference on field-programmable technology. IEEE; 2005, p. 215–22.
- [43] Khan FF, Ye A. An evaluation on the accuracy of the minimum-Width Transistor Area models in ranking the layout area of FPGA architectures. *ACM Trans Reconfigurable Technol Syst* 2018.
- [44] Elsaid K, Safar M, El-Kharashi MW. Optimized FPGA architecture for machine learning applications using posit multipliers. In: 2022 International conference on microelectronics. 2022, p. 50–3.